

My Cloth Client Libraries

Technical task documentation and interview guideProject: My Cloth AEM application | Date: 25 September 2026

1. Task Summary

The task was to configure global AEM client libraries for the My Cloth application. The client libraries provide reusable CSS and JavaScript to AEM pages, while the frontend build compiles Sass and JavaScript into deployable AEM assets.

Requirement

Implementation

Define mycloth.base

Configured in clientlib-base/.content.xml.

Define mycloth.site

Configured in clientlib-site/.content.xml and generated by clientlib.config.js.

Brand palette

Navy #0f172a, Gold #c9a84c, and neutral Sass tokens.

Typography

Avenir Next with Helvetica and Arial fallbacks.

Minification

GCC processors in AEM metadata and production Webpack minimizers.

2. What Changed

Clientlib metadata

```
categories="[mycloth.base]"
```

```
cssProcessor="[default:none,min:gcc]"
```

```
jsProcessor="[default:none,min:gcc]"
```

The same processor configuration is used by mycloth.site. The base library embeds selected AEM Core Component libraries and the grid library. The site library depends on mycloth.dependencies and contains the generated site assets.

Frontend styling

_variables.scss is the central design-token file. _base.scss applies the tokens to the page, links, and form controls. main.scss imports the variables, base styles, component styles, and site styles.

```
$color-navy: #0f172a;
```

```
$color-gold: #c9a84c;
```

```
$color-neutral-50: #f8fafc;
```

```
$font-family: "Avenir Next", "Helvetica Neue", Helvetica, Arial, sans-serif;
```

Build compatibility fixes

- Updated postcss-loader configuration to use the v8 postcssOptions API.

- Updated Maven frontend tooling from Node 16/npm 8 to Node 20/npm 10 because current dependencies use APIs such as toSorted().

3. Technical Architecture

Sass / JS source

ui.frontend/src

->

Webpack production build

npm run prod

->

AEM clientlib generator

clientlib.config.js

->

ui.apps clientlibs

/apps/mycloth/clientlibs

Webpack compiles Sass to CSS, bundles JavaScript, applies autoprefixing, and minifies production assets. The AEM clientlib generator copies the output into the ui.apps content package and writes clientlib metadata.

Page delivery flow

AEM page policy

->

Clientlib category

->

/etc.clientlibs proxy

->

Browser CSS/JS

The page policy includes the dependency and site categories. Page header/footer templates include the base category. AEM serves clientlibs through the proxy path so application code does not expose repository implementation paths directly.

4. Verification Performed

- 1 npm run prod completed successfully.
- 2 mvn clean verify completed successfully.
- 3 mvn clean install -PautoInstallPackage installed the package into local AEM author.

4 The following author endpoints returned generated assets successfully:

- /etc.clientlibs/mycloth/clientlibs/clientlib-base.css
- /etc.clientlibs/mycloth/clientlibs/clientlib-base.js
- /etc.clientlibs/mycloth/clientlibs/clientlib-site.css
- /etc.clientlibs/mycloth/clientlibs/clientlib-site.js

- 5 The served site CSS was minified and contained the expected Navy, Gold, neutral background, and typography values.

Scope note: Local author verification proves compilation, packaging, clientlib resolution, delivery, and minification. Publish runtime behavior still requires a publish instance or deployment pipeline verification.

5. How to Verify Locally

cd /Users/laxmanpeddi/MSM/mycloth

mvn clean install -PautoInstallPackage

- 1 Open <http://localhost:4502> and sign in.
- 2 Open a My Cloth page from AEM Sites.
- 3 Open browser DevTools, select Network, and reload.
- 4 Filter for clientlib and confirm the four category requests.
- 5 Inspect the CSS response for #0f172a, #c9a84c, and #f8fafc.
- 6 Use ?debugClientLibs=true when troubleshooting category resolution.

6. Interview Explanation

Short answer

"I configured two AEM client libraries. mycloth.base contains shared AEM component dependencies and global assets, while mycloth.site contains the application-specific CSS and JavaScript produced by Webpack. I added centralized Sass design tokens for the Navy, Gold, and neutral palette, configured GCC minification, fixed the PostCSS loader configuration for its installed version, and aligned Maven's Node version with the frontend dependencies. I validated the result by building the project, installing the package into local AEM author, and confirming the category-based clientlib URLs returned minified assets."

Likely questions and answers

Question

Answer

What is an AEM clientlib?

An AEM-managed collection of CSS, JavaScript, and resources. It has a category name, dependencies, processors, and a proxy URL used by pages.

Why separate base and site libraries?

Base contains shared platform/component assets. Site contains project-specific styling and behavior. This keeps responsibilities clear and makes dependencies reusable.

Why use categories?

Pages request stable category names instead of knowing physical asset filenames. AEM resolves dependencies and generates the final CSS/JS response.

How does minification happen?

Webpack production mode minifies the generated bundle, and the AEM clientlib metadata also declares GCC processors for clientlib processing.

How did you troubleshoot the build?

I followed the Maven error into the nested npm command, ran npm run prod directly, corrected the PostCSS v8 schema, then fixed the Node runtime mismatch revealed by toSorted().

How did you test it?

I ran the frontend build, Maven verification, package installation, direct clientlib endpoint checks, and browser Network inspection on local author.

What remains unverified?

Publish-specific runtime behavior, because no local publish instance was available. That should be checked in a publish SDK or deployment pipeline.

7. Key Files

- ui.apps/.../clientlib-base/.content.xml - base category and processors.
- ui.apps/.../clientlib-site/.content.xml - site category, dependency, and processors.
- ui.frontend/clientlib.config.js - frontend-to-AEM clientlib generation.
- ui.frontend/webpack.common.js - Sass, PostCSS, and asset pipeline.
- ui.frontend/src/main/webpack/site/_variables.scss - palette and typography tokens.
- pom.xml - Maven-managed Node/npm runtime.